

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Веб-технологии»

ОТЧЁТ ПО РАЗРАБОТКЕ БРАУЗЕРНОГО
РАСШИРЕНИЯ УРОВНЯ SENIOR

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Санкт-Петербург
2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ЗАДАНИЕ SENIOR.....	3
ОПИСАНИЕ АЛГОРИТМА РАБОТЫ.....	4

ЗАДАНИЕ SENIOR

Вариант задания: $(467036 \bmod 3) + 1 = 3$

Вариант №3 – контроль активности

Необходимо написать браузерное расширение, которое считает сколько времени пользователь провел на определенном сайте. Также должна быть страница настроек, где можно добавлять или убирать ранее добавленные URL с указанием времени, например 10 минут в день. Если времени на этом URL проведено больше, чем указано, то страницу больше невозможно открыть и пользователю выводится информация, что его лимит на сегодня исчерпан. Если он находится сейчас на этой странице, то тогда информация выводится моментально и просмотр страницы прекращается. Важно, чтобы указанные URL работали на весь сайт, то есть если пользователь укажет, например, ссылку на конкретное видео в YouTube, то ограничение будет все равно на весь YouTube.

ОПИСАНИЕ АЛГОРИТМА РАБОТЫ

Расширение было написано для браузеров на основе Chromium и Firefox (в браузере Safari возможности протестировать не было). Структура файлов:

- manifest.json - конфигурационный файл расширения;
- popup.html - файл разметки главного окна;
- popup.js - скрипт для реализации функционала главного окна;
- background.js - скрипт для работы в фоне (проверка лимитов, реагирование на открытие, закрытие и обновление вкладок и т.д.);
- settings - папка с файлами страницы настроек;
 - options.html - файл разметки страницы;
 - options.js - скрипт с функционалом (добавление и удаление ссылок, настройка времени, сохранение настроек и т.д.);
- blocked - папка с файлами страницы блокировки;
 - blocked.html - файл разметки;
 - blocked.js - скрипт с функционалом (определение заблокированного ресурса и вывод информации);

При нажатии на иконку расширения появляется небольшое окно, в котором присутствует кнопка "Открыть настройки", которая открывает страницу настроек расширения.

На странице настроек можно добавить новые ссылки для отслеживания активности, также указать максимальное количество минут в день, доступное для посещения сайта. При нажатии "Сохранить настройки" вся информация сохраняется в storage.sync.

Настройки Активности

Здесь вы можете настроить параметры контроля активности

Список URL:

github.com	✗	1488
vkvideo.ru	✗	0

Рисунок 1: Страница настроек

Главное окно расширения отображает информацию о текущем посещении страницы: время посещения и оставшееся время в минутах.

Контроль активности

Времени проведено на странице: 0.1 мин.

Осталось: 1487.9 мин.

Рисунок 2: Окно с временем для отслеживаемого сайта

Контроль активности

Времени проведено на странице: — мин.

Осталось: ∞ мин.

Рисунок 3: Окно для неотслеживаемого сайта

При достижении лимита по времени алгоритм будет постоянно перебрасывать пользователя на страницу с информацией о блокировке ресурса. Счётчик сбрасывается каждый день, поэтому для открытия страницы вновь нужно либо выждать время, либо удалить сайт из настроек.

Лимит времени исчерпан

Вы израсходовали установленный дневной лимит времени для сайта vkvideo.ru

А пока что можно зачлнить и подождать до завтра

Рисунок 4: Страница блокировки

Алгоритм работает на весь домен, а не на одну ссылку из домена. То есть при вводе ссылки <https://www.youtube.com/watch?v=dQw4w9WgXcQ&pp=ygUIcmIja3JvbGw%3D> в настройках алгоритм будет работать для всего домена youtube.com. Также минуты не будут считаться, если вкладка не находится в активном состоянии.

Расширение постоянно работает в фоне (background.js), чтобы слушать события открытия, обновления и закрытия вкладок, а также проверки на исчерпанные лимиты. Вся информация сохраняется в storage.sync (данные настроек) и в storage.local (данные об потраченных лимитах). Проверка происходит каждую секунду при срабатывании setInterval.

Исходный код manifest.json:

```
1 | {
2 |   "manifest_version": 3,
3 |   "name": "Контроль активности",
4 |   "version": "1.0",
5 |   "permissions": ["storage", "tabs", "activeTab"],
6 |   "author": "Пахомов Владимир Юрьевич 467036",
7 |   "description": "Расширение для контроля активности пользователя на определенных
сайтах",
8 |
9 |   "action": {
10 |     "default_popup": "popup.html",
11 |     "default_title": "Control Activity"
12 |   },
13 |
14 |   "background": {
15 |     "service_worker": "background.js",
16 |     "scripts": ["background.js"]
17 |   },
18 |
19 |   "options_ui": {
20 |     "page": "settings/options.html",
21 |     "open_in_tab": true
22 |   },
23 |
24 |   "browser_specific_settings": {
25 |     "gecko": {
26 |       "id": "@vladimir_pahomov_control_activity"
27 |     }
28 |   }
29 | }
```

Исходный код роруп.html:

```
1 | <!doctype html>
2 | <html lang="ru">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <link rel="stylesheet" href="style.css" />
7 |     <title>Контроль активности</title>
8 |   </head>
9 |   <body>
10 |     <h1>Контроль активности</h1>
11 |     <button id="open-options-btn">Открыть настройки</button>
12 |     <p>Времени проведено на странице: <span id="time-spent"></span> мин.</p>
13 |     <p>Осталось: <span id="time-left"></span> мин.</p>
14 |
15 |     <script src="popup.js"></script>
16 |   </body>
17 | </html>
```

Исходный код роруп.js:

```
1 | const browserAPI = typeof browser !== "undefined" ? browser : chrome;
2 |
3 | const openOptionsBtn = document.getElementById("open-options-btn");
4 | const timeSpentSpan = document.getElementById("time-spent");
5 | const timeLeftSpan = document.getElementById("time-left");
6 |
7 | openOptionsBtn.addEventListener("click", () => {
8 |   if (browserAPI.runtime.openOptionsPage) {
9 |     browserAPI.runtime.openOptionsPage();
10 |   } else {
11 |     window.open(browserAPI.runtime.getURL("settings/options.html"));
12 |   }
13 | });
14 |
15 | function getDomain(url) {
16 |   if (!url) return null;
17 |
18 |   let cleanUrl = url.trim();
19 |   if (!cleanUrl.includes("://")) {
20 |     cleanUrl = "http://" + cleanUrl;
21 |   }
22 |
23 |   try {
24 |     const parsed = new URL(cleanUrl);
25 |     return parsed.hostname.replace(/^www\./, "").toLowerCase();
26 |   } catch (e) {
27 |     return null;
28 |   }
29 | }
30 |
31 | function findMatchRule(visitedDomain, urls) {
32 |   if (!visitedDomain || !Array.isArray(urls)) return null;
33 |   return urls.find((rule) => {
34 |     const limitDomain = getDomain(rule.url);
35 |     if (!limitDomain) return false;
36 |     return (
37 |       visitedDomain === limitDomain || visitedDomain.endsWith("." + limitDomain)
38 |     );
39 |   });
40 | }
41 |
42 | async function updatePopup() {
43 |   browserAPI.tabs.query({ active: true, currentWindow: true }, async (tabs) => {
44 |     if (tabs.length === 0) return;
45 |   });
46 | }
```



```

46 |         const domain = getDomain(tabs[0].url);
47 |         if (!domain) {
48 |             timeSpentSpan.textContent = "-";
49 |             timeLeftSpan.textContent = "-";
50 |             return;
51 |         }
52 |
53 |         const syncData = await browserAPI.storage.sync.get("urls");
54 |         const localData = await browserAPI.storage.local.get("stats");
55 |
56 |         const urls = syncData.urls || [];
57 |         const stats = localData.stats || {};
58 |
59 |         const rule = findMatchRule(domain, urls);
60 |
61 |         if (rule) {
62 |             const limitDomain = getDomain(rule.url);
63 |             const spentSeconds = stats[limitDomain] || 0;
64 |             const spentMinutes = (spentSeconds / 60).toFixed(1);
65 |             const limitMinutes = rule.time;
66 |             const leftMinutes = Math.max(0, limitMinutes - spentSeconds / 60).toFixed(
67 |                 1,
68 |             );
69 |
70 |             timeSpentSpan.textContent = spentMinutes;
71 |             timeLeftSpan.textContent = leftMinutes;
72 |         } else {
73 |             timeSpentSpan.textContent = " - ";
74 |             timeLeftSpan.textContent = "∞";
75 |         }
76 |     });
77 | }
78 |
79 | updatePopup();

```

Исходный код background.js:

```

1 |     const browserAPI = typeof browser !== "undefined" ? browser : chrome;
2 |
3 |     let activeTabId = null;
4 |     let activeDomain = null;
5 |     let timerInterval = null;
6 |
7 |     function getDomain(url) {
8 |         if (!url) return null;
9 |
10 |         let cleanUrl = url.trim();
11 |         if (!cleanUrl.includes("://")) {
12 |             cleanUrl = "http://" + cleanUrl;
13 |         }
14 |
15 |         try {
16 |             const parsed = new URL(cleanUrl);
17 |             return parsed.hostname.replace(/^www\.\/, "").toLowerCase();
18 |         } catch (e) {
19 |             return null;
20 |         }
21 |     }
22 |
23 |     function findMatchRule(visitedDomain, urls) {
24 |         if (!visitedDomain || !Array.isArray(urls)) return null;
25 |         return urls.find((rule) => {
26 |             const limitDomain = getDomain(rule.url);
27 |             if (!limitDomain) return false;
28 |             return (
29 |                 visitedDomain === limitDomain || visitedDomain.endsWith("." + limitDomain)
30 |             );
31 |         });

```

```

32 |     }
33 |
34 |     async function checkLimitAndBlock(tabId, domain) {
35 |         if (!domain) return;
36 |
37 |         const syncData = await browserAPI.storage.sync.get("urls");
38 |         const localData = await browserAPI.storage.local.get([
39 |             "stats",
40 |             "lastResetDate",
41 |         ]);
42 |
43 |         const urls = syncData.urls || [];
44 |         const stats = localData.stats || {};
45 |         const lastResetDate = localData.lastResetDate;
46 |
47 |         const today = new Date().toDateString();
48 |         if (lastResetDate !== today) {
49 |             await browserAPI.storage.local.set({ stats: {}, lastResetDate: today });
50 |         }
51 |
52 |         const rule = findMatchRule(domain, urls);
53 |         if (rule) {
54 |             const limitDomain = getDomain(rule.url);
55 |             const spentSeconds = stats[limitDomain] || 0;
56 |             const allowedSeconds = rule.time * 60;
57 |
58 |             if (spentSeconds >= allowedSeconds) {
59 |                 blockTab(tabId, limitDomain);
60 |             }
61 |         }
62 |     }
63 |
64 |     function blockTab(tabId, domain) {
65 |         const blockUrl =
66 |             browserAPI.runtime.getURL("blocked/blocked.html") +
67 |             `?domain=${encodeURIComponent(domain)}`;
68 |         browserAPI.tabs.update(tabId, { url: blockUrl });
69 |     }
70 |
71 |     function startTracking() {
72 |         if (timerInterval) clearInterval(timerInterval);
73 |
74 |         timerInterval = setInterval(async () => {
75 |             if (!activeDomain || !activeTabId) return;
76 |
77 |             const syncData = await browserAPI.storage.sync.get("urls");
78 |             const localData = await browserAPI.storage.local.get([
79 |                 "stats",
80 |                 "lastResetDate",
81 |             ]);
82 |
83 |             const urls = syncData.urls || [];
84 |             let stats = localData.stats || {};
85 |             const today = new Date().toDateString();
86 |
87 |             if (localData.lastResetDate !== today) {
88 |                 stats = {};
89 |                 await browserAPI.storage.local.set({ stats: {}, lastResetDate: today });
90 |             }
91 |             const rule = findMatchRule(activeDomain, urls);
92 |
93 |             if (rule) {
94 |                 const limitDomain = getDomain(rule.url);
95 |                 stats[limitDomain] = (stats[limitDomain] || 0) + 1;
96 |
97 |                 await browserAPI.storage.local.set({ stats });
98 |
99 |                 if (stats[limitDomain] >= rule.time * 60) {
100 |                     blockTab(activeTabId, limitDomain);
101 |                 }
102 |             }

```

```

103     }, 1000);
104 }
105
106 async function updateActiveTab() {
107     browserAPI.tabs.query({ active: true, currentWindow: true }, async (tabs) => {
108         if (tabs.length === 0) {
109             activeTabId = null;
110             activeDomain = null;
111             return;
112         }
113         const tab = tabs[0];
114         activeTabId = tab.id;
115         activeDomain = getDomain(tab.url);
116
117         if (activeDomain) {
118             await checkLimitAndBlock(activeTabId, activeDomain);
119         }
120     });
121 }
122
123 browserAPI.tabs.onActivated.addListener(updateActiveTab);
124 browserAPI.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
125     if (changeInfo.status === "complete" || changeInfo.url) {
126         updateActiveTab();
127     }
128 });
129
130 browserAPI.windows.onFocusChanged.addListener((windowId) => {
131     if (windowId === browserAPI.windows.WINDOW_ID_NONE) {
132         activeDomain = null;
133         activeTabId = null;
134     } else {
135         updateActiveTab();
136     }
137 });
138
139 startTracking();

```

Исходный код settings/options.html:

```

1  <!doctype html>
2  <html lang="ru">
3      <head>
4          <meta charset="UTF-8" />
5          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6
7          <title>Настройки Активности</title>
8      </head>
9      <body>
10         <style>
11             ul {
12                 list-style: none;
13             }
14             fieldset {
15                 width: 600px;
16             }
17             input[type="number"] {
18                 margin-left: 2rem;
19                 width: 4rem;
20             }
21             button {
22                 margin: 0.5rem;
23             }
24         </style>
25         <h1>Настройки Активности</h1>
26         <p>Здесь вы можете настроить параметры контроля активности</p>
27         <fieldset id="url-list">
28             <legend>Список URL:</legend>

```

```

29 |         <ul id="url-items">
30 |             <!-- Список URL будет динамически заполняться -->
31 |         </ul>
32 |     </fieldset>
33 |     <div class="btns-container">
34 |         <button id="add-url-btn">Добавить новый URL</button>
35 |         <button id="save-btn">Сохранить настройки</button>
36 |     </div>
37 |     <script src="options.js"></script>
38 | </body>
39 | </html>

```

Исходный код settings/options.js:

```

1 | const browserAPI = typeof browser !== "undefined" ? browser : chrome;
2 |
3 | const urlList = document.getElementById("url-items");
4 | const saveBtn = document.getElementById("save-btn");
5 | const addUrlBtn = document.getElementById("add-url-btn");
6 |
7 | class UrlItem {
8 |     constructor(url, time) {
9 |         this.url = url;
10 |         this.time = time;
11 |         this.li = document.createElement("li");
12 |         this.li.className = "url-item";
13 |
14 |         const urlInput = document.createElement("input");
15 |         urlInput.type = "text";
16 |         urlInput.placeholder = "Ссылка";
17 |         urlInput.value = url;
18 |
19 |         const deleteBtn = document.createElement("button");
20 |         deleteBtn.textContent = "✖";
21 |         deleteBtn.addEventListener("click", () => this.li.remove());
22 |
23 |         const timeInput = document.createElement("input");
24 |         timeInput.type = "number";
25 |         timeInput.min = "0";
26 |         timeInput.placeholder = "Время в минутах";
27 |         timeInput.value = String(time) || "0";
28 |
29 |         this.li.appendChild(urlInput);
30 |         this.li.appendChild(deleteBtn);
31 |         this.li.appendChild(timeInput);
32 |     }
33 | }
34 |
35 | browserAPI.storage.sync.get("urls").then((result) => {
36 |     const urls = result.urls || [];
37 |     urls.forEach(({ url, time }) => {
38 |         const urlItem = new UrlItem(url, time);
39 |         urlList.appendChild(urlItem.li);
40 |     });
41 | });
42 |
43 | function saveSettings() {
44 |     const urls = [];
45 |     urlList.querySelectorAll(".url-item").forEach((li) => {
46 |         const urlInput = li.querySelector("input[type='text']");
47 |         const timeInput = li.querySelector("input[type='number']");
48 |         const url = urlInput.value.trim();
49 |         const time = parseInt(timeInput.value, 10) || 0;
50 |         if (url !== "") {
51 |             urls.push({ url, time });
52 |         }
53 |     });
54 |     browserAPI.storage.sync.set({ urls: urls });

```

```

55 |     }
56 |
57 |     function addUrl() {
58 |         const newUrlItem = new UrlItem("", 0);
59 |         urlList.appendChild(newUrlItem.li);
60 |     }
61 |
62 |     saveBtn.addEventListener("click", saveSettings);
63 |     addUrlBtn.addEventListener("click", addUrl);

```

Исходный код blocked/blocked.html:

```

1 | <!doctype html>
2 | <html lang="ru">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>Доступ ограничен</title>
7 |   </head>
8 |   <body>
9 |     <h1>Лимит времени исчерпан</h1>
10 |     <p>
11 |       Вы израсходовали установленный дневной лимит времени для сайта
12 |       <span id="domain-name"></span>
13 |     </p>
14 |     <p>А пока что можно зачиститься и подождать до завтра</p>
15 |     <script src="blocked.js"></script>
16 |   </body>
17 | </html>

```

Исходный код blocked/blocked.js:

```

1 | const urlParams = new URLSearchParams(window.location.search);
2 | const domain = urlParams.get("domain");
3 | const domainName = document.getElementById("domain-name");
4 | domainName.textContent = domain || "этого песчуса";

```